

CSS Flexbox

Ihr Dozent: Robert Reich

Nützliche Resourcen - flexbox

- https://kulturbanause.de/blog/einfuhrung-in-das-flexbox-modell-von-css/

- https://www.w3schools.com/css/css3_flexbox.asp
- https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout/Flexbox

Flexbox

Mit der Flexbox kann man unkompliziert variable Layouts erstellen. Das Modell arbeitet in zwei Achsen, Haupt und Kreuzachse und ist daher sehr gut geeignet responsive Websites ohne Kopfschmerzen umzusetzen.

Flexbox

Flexbox arbeitet mit einem Container-Element – dem. sog. Flex-Container – und den darin enthaltenen Kind-Elementen der 1. Ebene – den sog. Flex-Items. Die Flexbox wirkt genau eine Ebene tief. Weitere Verschachtelungen werden ignoriert.

Flexbox

```
<div class="flex-container">
  <div class="flex-item"></div>
  <div class="flex-item"></div>
  <div class="flex-item"></div>
  <div class="flex-item"></div>
</div>
```

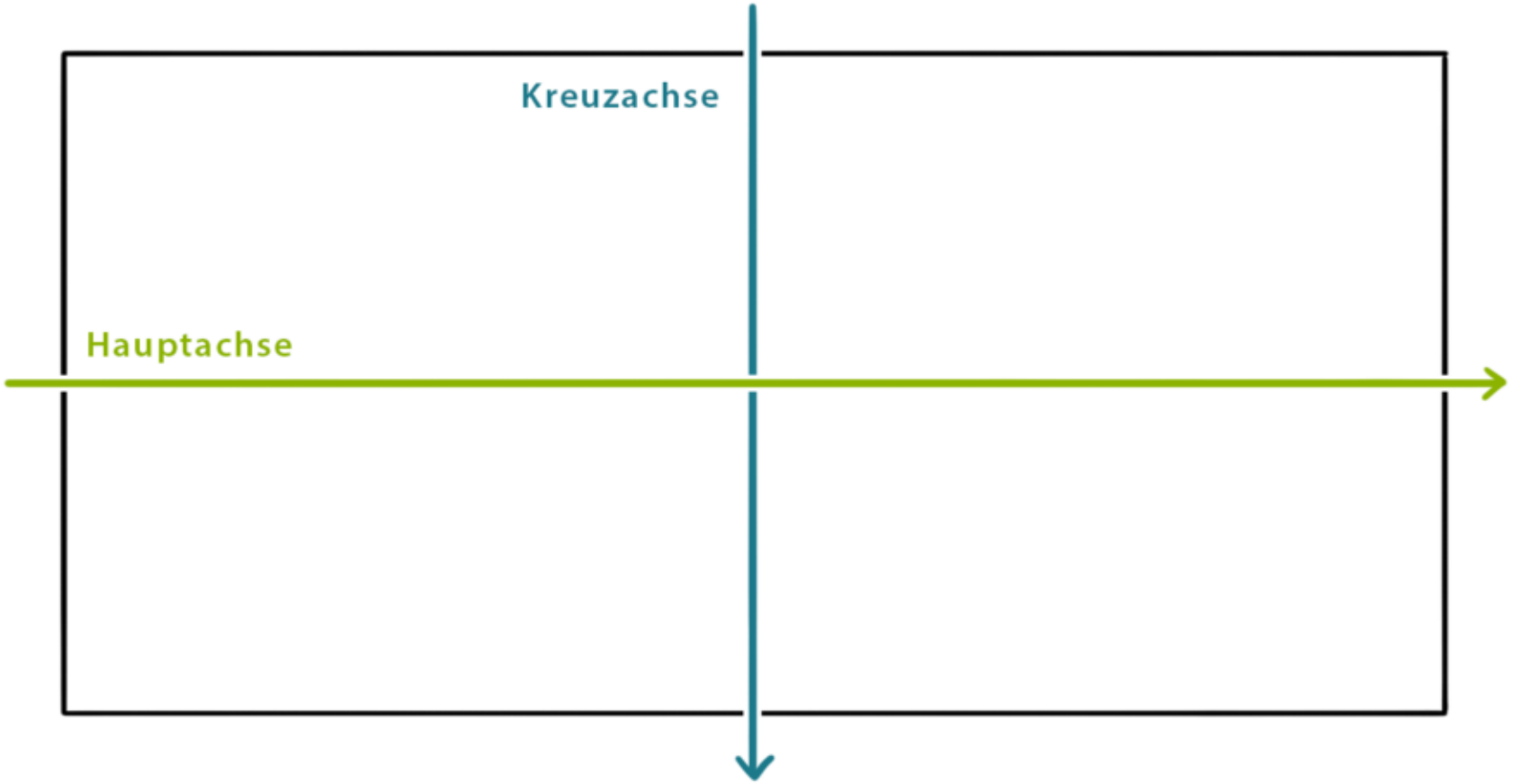
Flexbox

Mit display: flex; auf dem Hauptcontainer wird die Flexbox aktiviert.

```
.flex-container {
  display:flex;
}
```

Flexbox

Haupt-Achse & Kreuz-Achse der Flexbox



Flexbox

Die Items können nun mit Hilfe der Container-Eigenschaften justify-content und align-items auf den beiden Achsen positioniert werden. Die Ausrichtung der Hauptachse kann mit flex-direction verändert werden.

justify-content steuert die Items auf der Haupt-Achse
align-items steuert die Items auf der Kreuz-Achse
flex-direction dreht die Haupt-Achse

Flexbox

Wenn lediglich mit display:flex; die Flexbox eingeschaltet wurde, wirken die Standardwerte für justify-content, align-items und flex-direction.

Flexbox

Der Standardwert für flex-direction ist row – von links nach rechts
Der Standardwert für justify-content ist flex-start: Die Items stehen am Anfang der Hauptachse, was normalerweise einer Positionierung links entspricht.
Der Standardwert für align-items ist stretch: Alle Items sind daher standardmäßig so hoch wie der Flex-Container.

Flexbox - flex-direction, Die Achsen drehen

Wie bereits beschrieben, läuft die Hauptachse standardmäßig von links nach rechts und die Kreuzachse von oben nach unten. Mit der Container-Eigenschaft flex-direction können die Achsen gedreht werden.

Flexbox - flex-direction

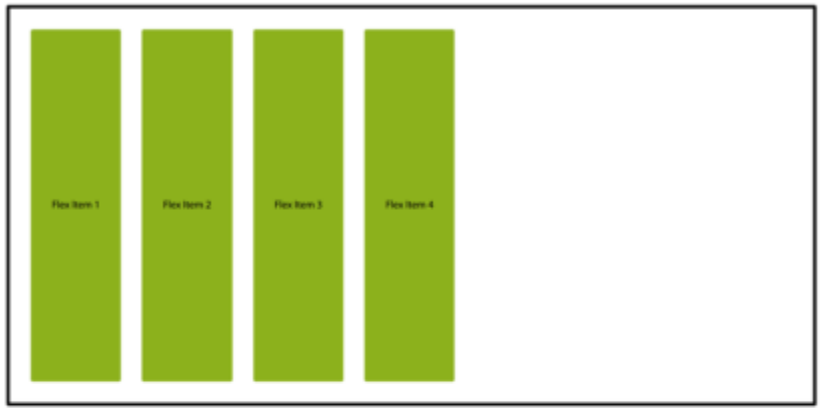
```
.flex-container {
  display: flex;
  flex-direction: row; /* Standardwert */
}
```

Flexbox - flex-direction

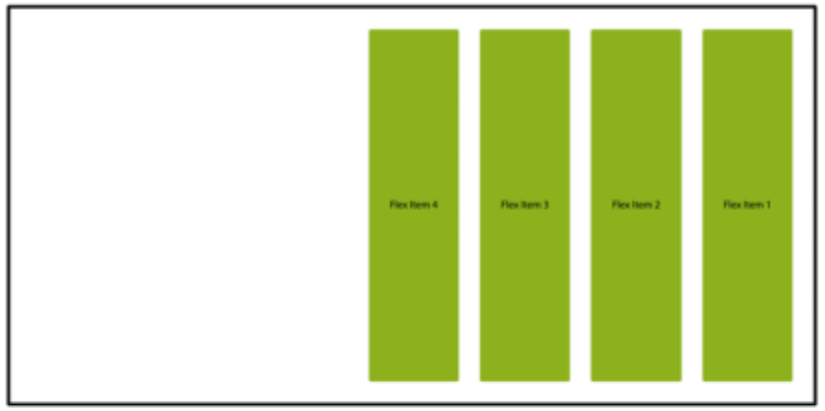
flex-direction: row (von links nach rechts = Standardwert)
flex-direction: column (von oben nach unten)
flex-direction: row-reverse (von rechts nach links)
flex-direction: column-reverse (von unten nach oben)

Flexbox - flex-direction

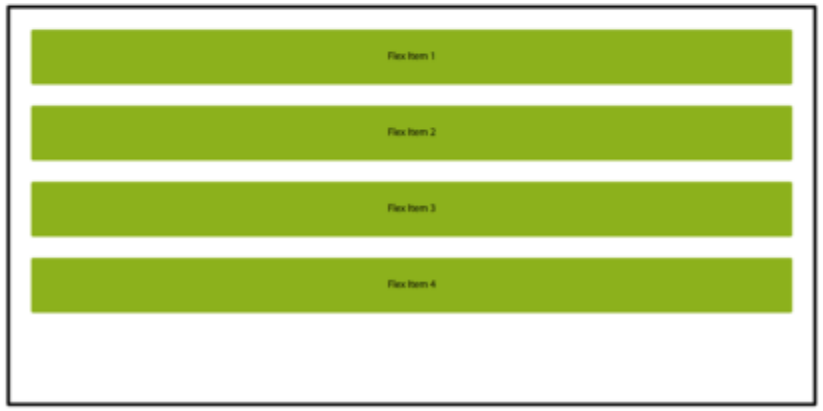
flex-direction: row



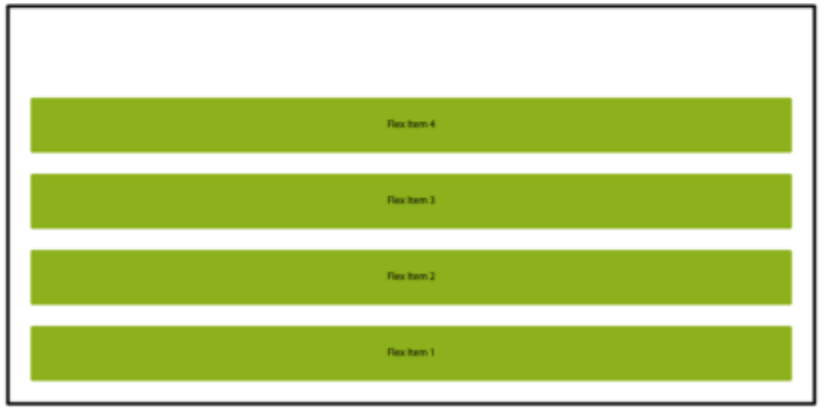
flex-direction: row-reverse



flex-direction: column



flex-direction: column-reverse

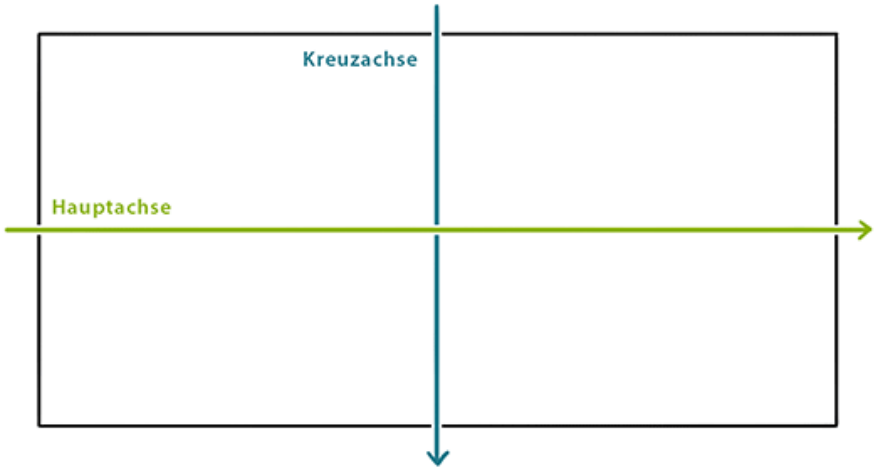


Flexbox - flex-direction

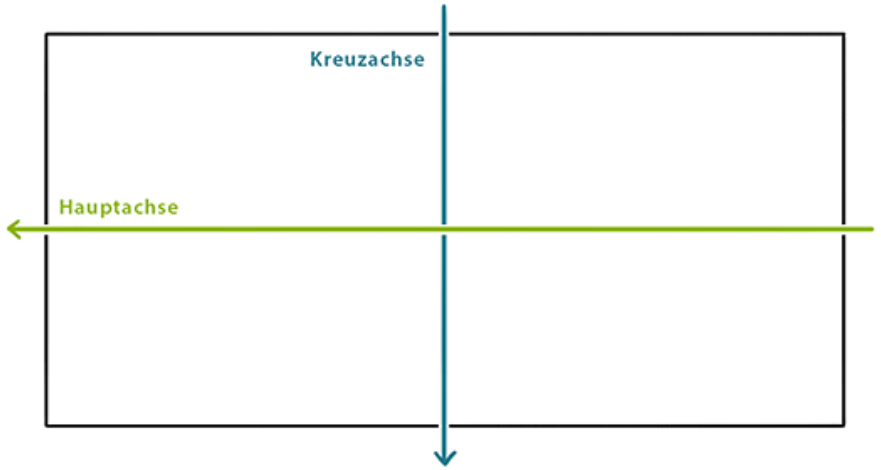
Wichtig ist, dass mit flex-direction die beiden Achsen nicht gemeinsam gedreht werden, sondern nur die Hauptachse. Wie die Kreuzachse im Verhältnis zur Hauptachse positioniert ist, zeigt die folgende Grafik.

Flexbox - flex-direction

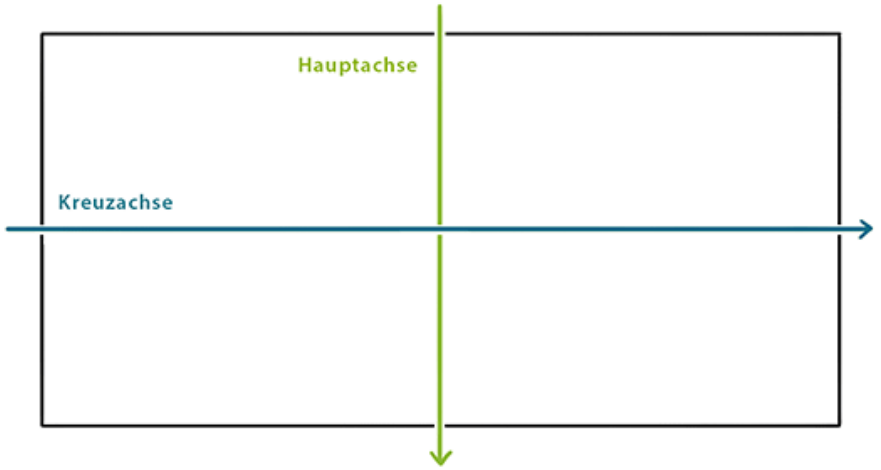
flex-direction: row;



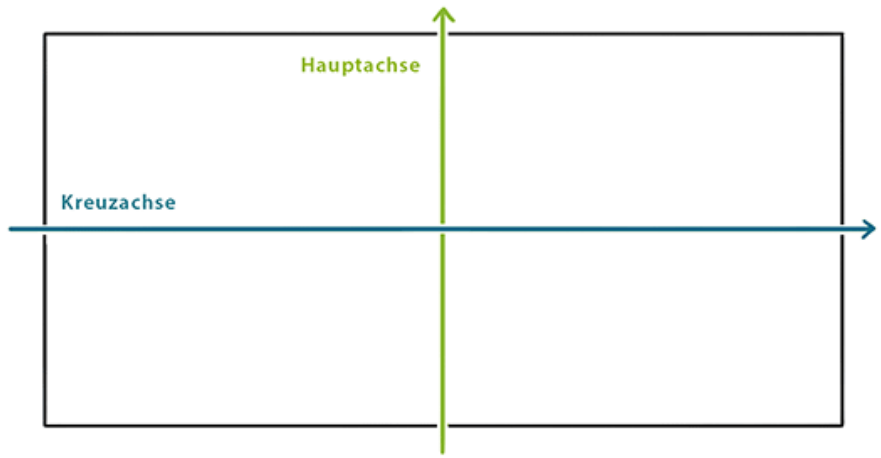
flex-direction: row-reverse;



flex-direction: column;



flex-direction: column-reverse;



Flexbox - justify-content

Items auf der Hauptachse positionieren.

```
.flex-container {
  display: flex;
  justify-content: flex-start; /* Standardwert */
}
```

Flexbox - justify-content

justify-content: flex-start (Am Anfang der Hauptachse)
justify-content: center (In der Mitte der Hauptachse)
justify-content: flex-end (Am Ende der Hauptachse)
justify-content: space-between (Das 1. Flex-Item ist ganz am Anfang der Achse, das letzte ganz am Ende. Alle anderen Items werden gleichmäßig verteilt. Es entstehen also Abstände zwischen den Items.)
justify-content: space-around (Die Flex-Items werden wie bei space-between verteilt, allerdings entsteht auch vor dem 1. Item und nach dem letzten Item ein Abstand)
justify-content: space-evenly (Die Flex-Items werden wie bei den beiden vorherigen Methoden verteilt, allerdings werden die Abstände einheitlich gewählt)

Flexbox - justify-content

justify-content: flex-start



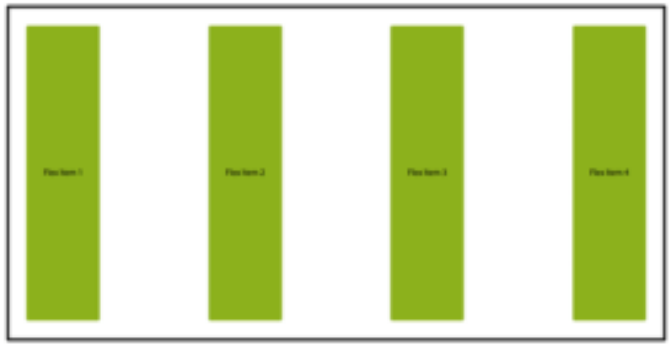
justify-content: center



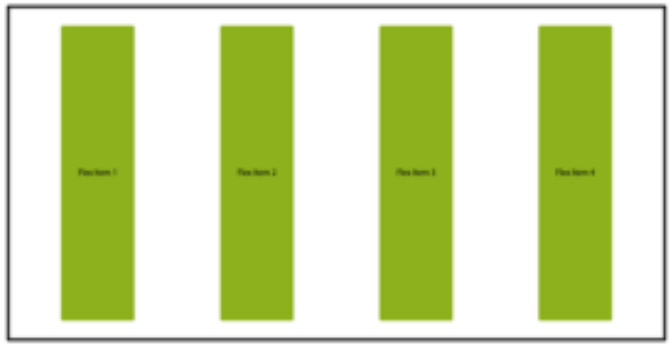
justify-content: flex-end



justify-content: space-between



justify-content: space-around



justify-content: space-evenly



Flexbox – align-items

Auf der Kreuzachse werden die Items normalerweise mit dem Wert stretch positioniert: Alle Items sind so hoch wie der Flex-Container.

```
.flex-container {
  display: flex;
  align-items: stretch; /* Standardwert */
}
```

Flexbox – align-items

align-items: flex-start (Am Anfang der Kreuzachse)
align-items: center (In der Mitte der Kreuzachse)
align-items: flex-end (Am Ende der Kreuzachse)
align-items: baseline (Die Flex-Items werden so ausgerichtet, dass die Grundlinie der Schrift bündig ist. Nur sichtbar bei unterschiedlichen Schriftgrößen)
align-items: stretch (Alle Flex-Items sind so hoch wie die Flexbox)

Flexbox – align-items

align-items: flex-start



align-items: center



align-items: flex-end



align-items: baseline



align-items: stretch

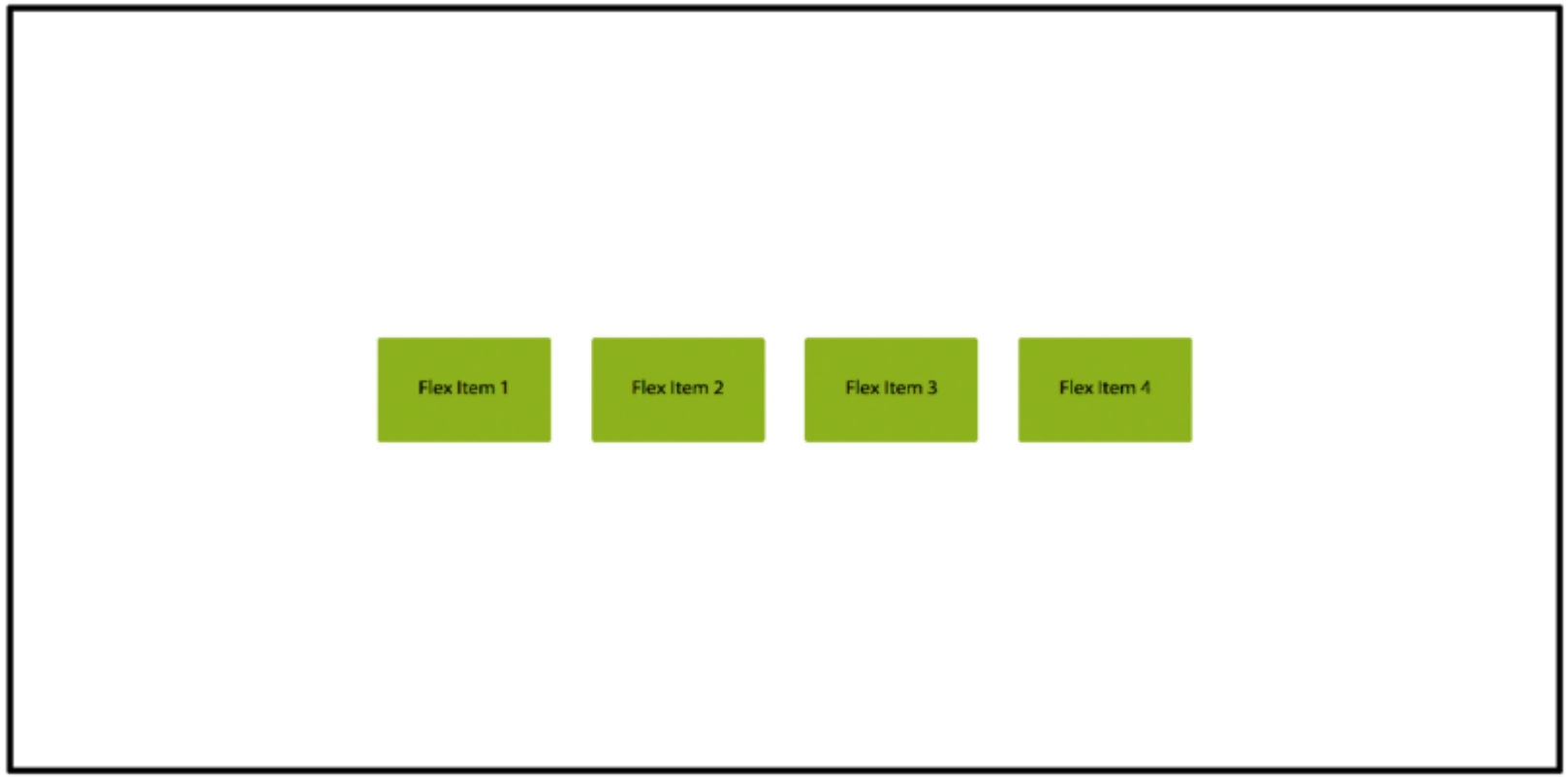


Flexbox

Beispiel – horizontal und vertikal zentrieren

```
.flex-container {
  display: flex;
  justify-content: center;
  align-items: center;
}
```

Flexbox – Center horizontal / vertikal



Flexbox - Eigenschaften der Flex-Items

Neben der Positionierung über den Flex-Container können auch die Items selbst beeinflusst werden. Dies kann einerseits mit den klassischen CSS-Angaben für Größe und Box-Modell erledigt werden. Darüber hinaus existieren in der CSS Flexbox die Eigenschaften flex-basis, flex-grow und flex-shrink zur Beeinflussung der Größe eines Items sowie order zur Veränderung der Reihenfolge von Items.

Flexbox - Eigenschaften der Flex-Items

flex-grow: 0 (Items dürfen nicht größer werden als festgelegt)
flex-shrink: 1 (Items dürfen kleiner werden als festgelegt)
flex-basis: auto (Die Basis-Größe der Items wird automatisch festgelegt – also anhand der Angaben height, width bzw. auf Grundlage des enthaltenen Inhalts)
order: 0 (Die Reihenfolge entspricht der des HTML-Codes)

Flexbox - Eigenschaften der Flex-Items

```
/* Kurzschreibweise */
.flex-item {
  flex: 0 1 auto;
}

/* Lange Schreibweise */
.flex-item {
  flex-grow: 0;
  flex-shrink: 1;
  flex-basis: auto
}
```

Flexbox - Eigenschaften der Flex-Items

flex-grow, flex-shrink und flex-basis – Flexibilität für Items festlegen

Mit den verschiedenen flex-Werten kann das gewünschte Layout innerhalb des Containers hergestellt werden. Die folgenden Beispiele mit jeweils vier Items zeigen was möglich ist.

Mit flex-grow: 1; erlauben wir dem 2. Item sich auszudehnen.

```
flex-item-2 {
  flex-grow: 1;
}
```

Flexbox – Beispiel flex-grow item 2



Flexbox - order

order – Die Reihenfolge von Items beeinflussen

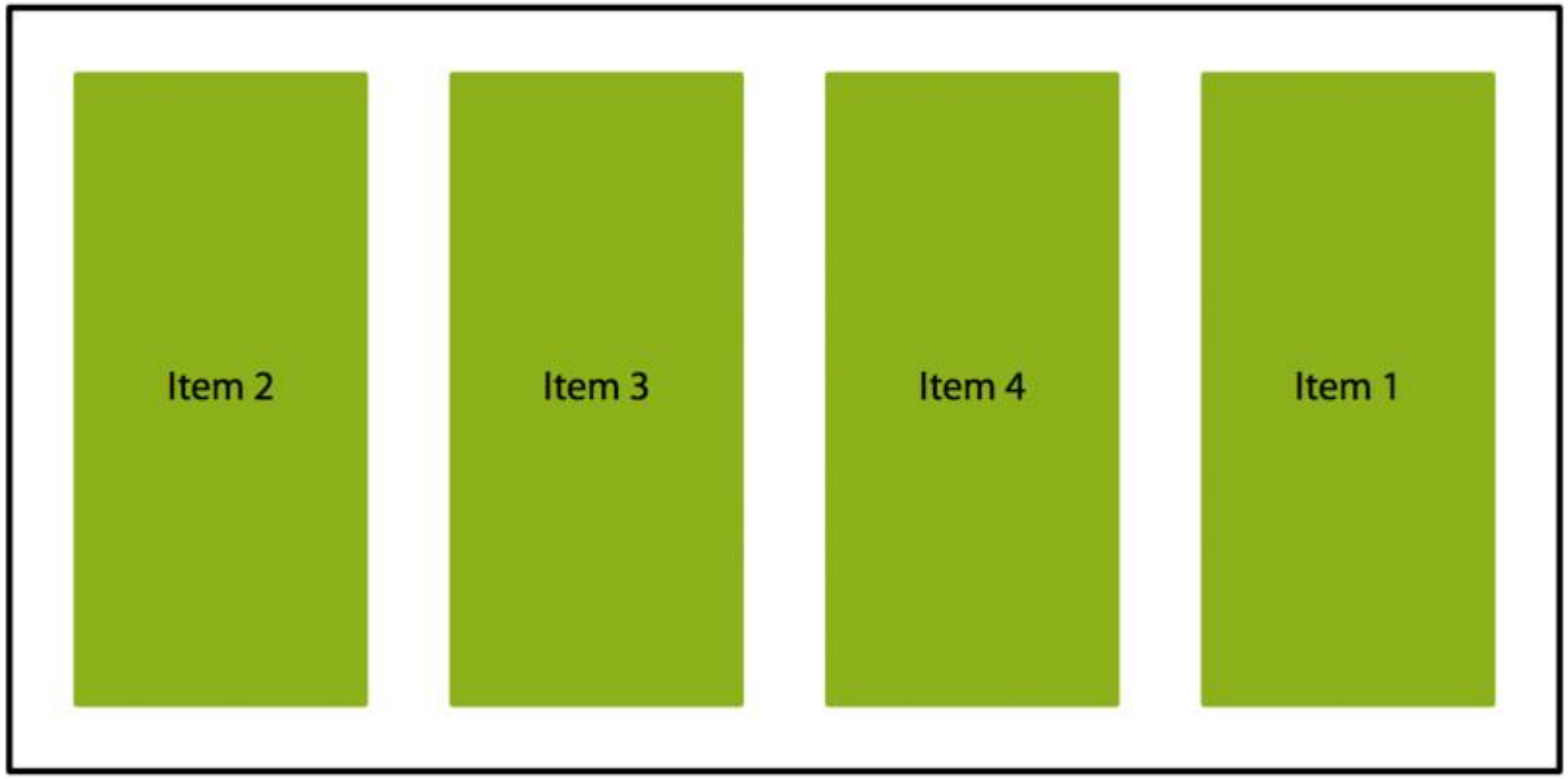
Normalerweise werden die Items in der Reihenfolge angezeigt, in der sie im HTML-Code notiert wurden. Mit der Eigenschaft order kann diese Reihenfolge geändert werden, ohne dass der HTML-Code angepasst werden muss.

```
.flex-item-1 {
  order: 4;
}
```

Flexbox – Order Beispiel

```
.flex-item-1 {
  order: 4;
}
.flex-item-2 {
  order: 1;
}
.flex-item-3 {
  order: 2;
}
.flex-item-4 {
  order: 3;
}
```

Flexbox – Beispiel flex-grow item 2



Flexbox – flex-wrap

Mehrspaltigkeit über das Container-Element ermöglichen

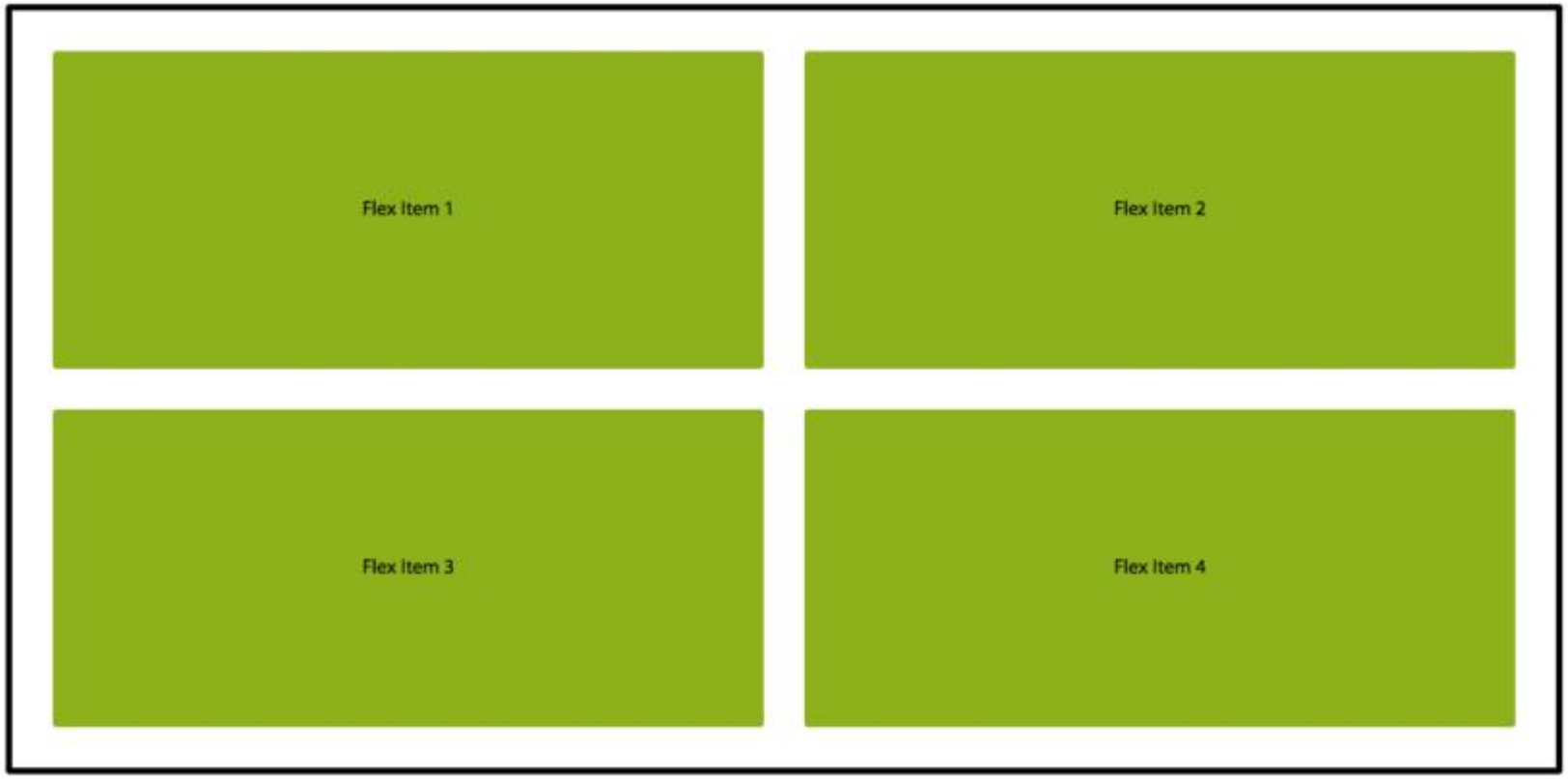
Normalerweise werden die Items innerhalb der Flexbox linear nebeneinander oder untereinander auf den Achsen positioniert. Wenn mehrere Items von der Abmessung her nicht in den Container passen, kann mit flex-grow und -shrink festgelegt werden was passieren soll. Mit der Container-Eigenschaft flex-wrap kann festgelegt werden, dass Items in mehrere Spalten und Zeilen umbrechen.

Flexbox – flex-wrap

```
.flex-container {
  display: flex;
  flex-wrap: wrap; /* Standardwert = nowrap */
}

.flex-item {
  flex-basis: 50%;
  flex-grow: 1;
  flex-shrink: 1;
}
```

Flexbox – Beispiel flex-grow item 2



Flexbox – gap

Die Eigenschaft gap definiert den Abstand zwischen items in einer Flexbox. Bei drei Untercontainern in einer horizontalen Box würde gap: 16px zwischen Box 1 & 2 sowie zwischen Box 2 & 3 einen Abstand von 16 Pixeln schaffen, nicht aber an den äußeren Ränder.

```
.flexbox {
  display: flex;
  gap: 16px;

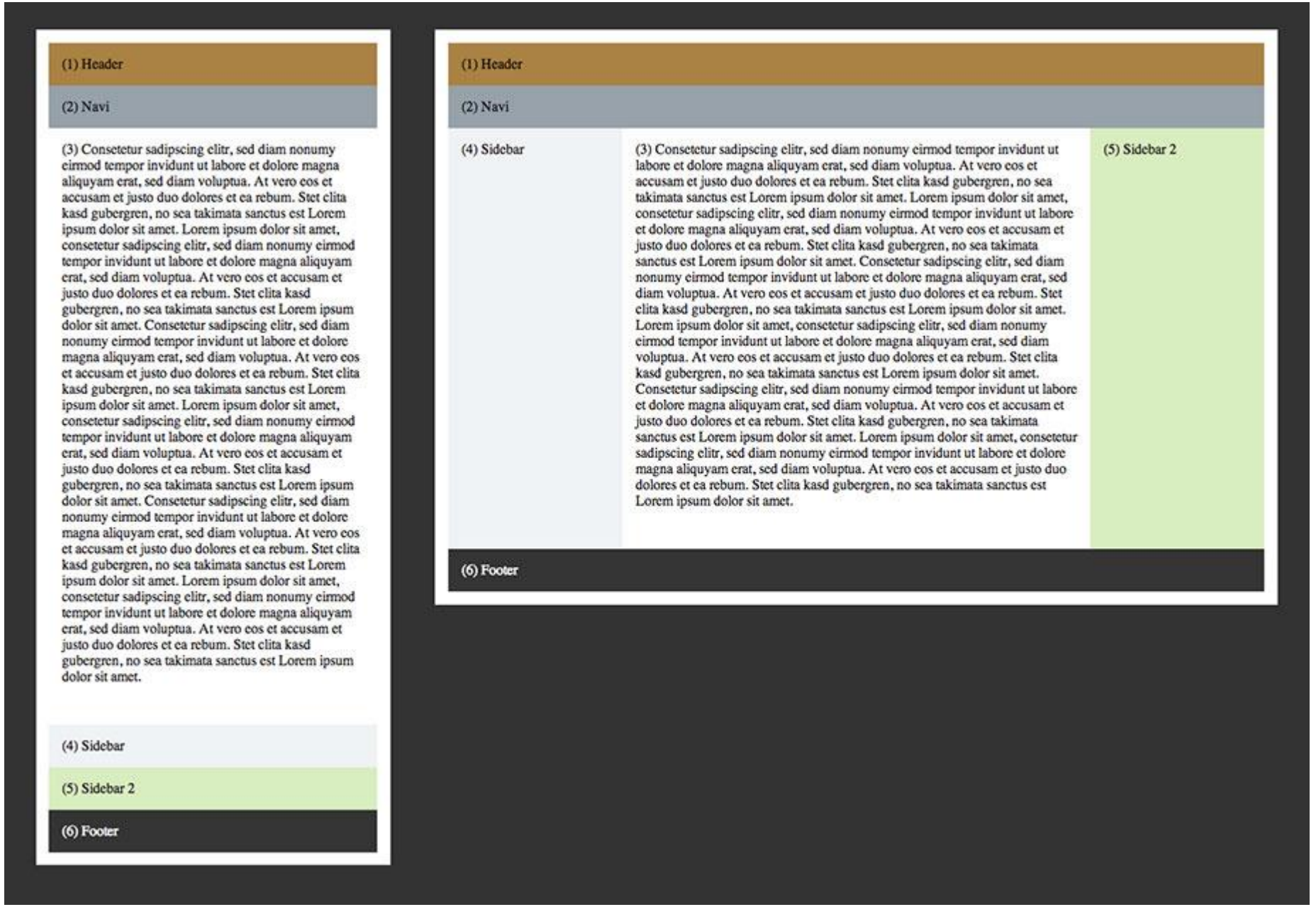
.flex-item {
  width: 33.3334%;
}
```

Responsive Design mit CSS Flexbox

Mit den nun kennengelernten Eigenschaften und einem Media Query lässt sich unkompliziert kleines Praxisbeispiel erstellen.

Die folgende Website ist so aufgebaut, dass in der Smartphone-Ansicht (und im HTML-Quellcode) die beiden Sidebars nach dem Hauptinhalt angezeigt werden. In der Desktop-Ansicht wird mit Hilfe von Flexbox der Hauptinhalt zwischen den Sidebars dargestellt.

Wir erreichen so gleichzeitig die perfekte Darstellung und den semantisch sinnvollsten HTML-Code.


(1) Header
(2) Navi
(3) Consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.
(4) Sidebar
(5) Sidebar 2
(6) Footer
(1) Header
(2) Navi
(4) Sidebar
(3) Consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet. Consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit amet.
(5) Sidebar 2
(6) Footer

Responsive Design mit CSS Flexbox

```
@media screen and (min-width: 800px) { /* Screen-Breite kleiner/gleich als 800px */
  .content {
    display: flex;
  }
  .main {
    flex: 3;
    order: 2;
  }
  .sidebar {
    flex: 1;
    order: 1;
  }
  .sidebar2 {
    flex: 1;
    order: 3;
  }
}
```

CSS Flexbox Beispiele

Responsive Demo:
https://media.kulturbanause.de/2013/07/flexbox-7-media-queries.html

Fazit: Was brauchen wir wirklich?

display: flex;

/* row ist Standard, muss also horizontal nicht angegeben werden, nur bei vertikaler Ausrichtung (column) */
flex-direction: row;

/* optional aber hilfreich */
justify-content:
flex-start, center, flex-end, space-between, space-around, space-evenly

/* Standardwert stretch oder zum vertikalen zentrieren: center */
align-items: stretch, center

Flexbox Übersicht

Haupt- / Obercontainer

display: flex
flex-direction: row (default) oder column
justify-content: flex-start (default), center, flex-end, space-between, space-around, space-evenly
align-items: flex-start, center, flex-end, baseline, stretch (default)
flex-wrap: wrap oder nowrap (default)

Untercontainer

order: 1
flex-grow: 0
flex-shrink: 1
flex-basis: auto